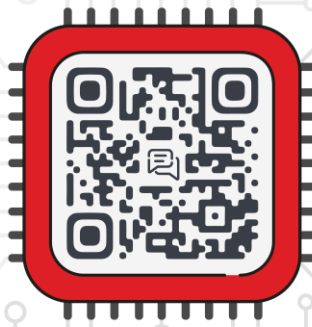




نظري

2

22

2640 SP

كلية الهندسة المعلوماتية

السنة الرابعة - النظم والشبكات

UDP Sockets ,
Threads

د. غسان شحود



RB Informatcs محتوي مجاني غير مخصص للبيع التجاري
10/11/2025

برمجة تطبيقات شبكية

السلام عليكم ورحمة الله وبركاته

تحدثنا في المحاضرة السابقة عن Tcp Sockets و Tcp Sockets و نموذج server-client وكيفية تمثيله في لغة Java ، وما هي أهم الكلاسات والتوابع التي تساعدنا في ذلك. في محاضرتنا هذه سنتناول شرح UDP Sockets وكيفية تمثيلها و سنتذكر مفاهيم عديدة في الـ Threading.

UDP (User Datagram Protocol)

هو بروتوكول نقل بيانات ، سريع و غير موجه بالاتصال عبر الانترنت ، أي أنه يمكن أن يرسل بيانات إلى الوجهة دون إنشاء اتصال مسبق ، وهو غير موثوق (unreliable) أي أنه لا يضمن وصول الداتا بالترتيب و إذا تم فقدان جزء منها فهو لن يقوم بإعادة إرسالها ، لا يوجد hand shaking . إنه يأخذ البيانات القادمة من طبقة الـ Application ثم يضعها في Datagrams ويضيف لها header ويرسلها عبر الشبكة.

version	header length	type of service	datagram length	
identification			flags	fragment offset
time-to-live (TTL)	protocol	header checksum		
source address				
destination address				
options				
source port (0–65,535)		destination port (0–65,535)		
combined length of data and UDP header (8–65,515)		Checksum		
data				

IP Header

UDP Header

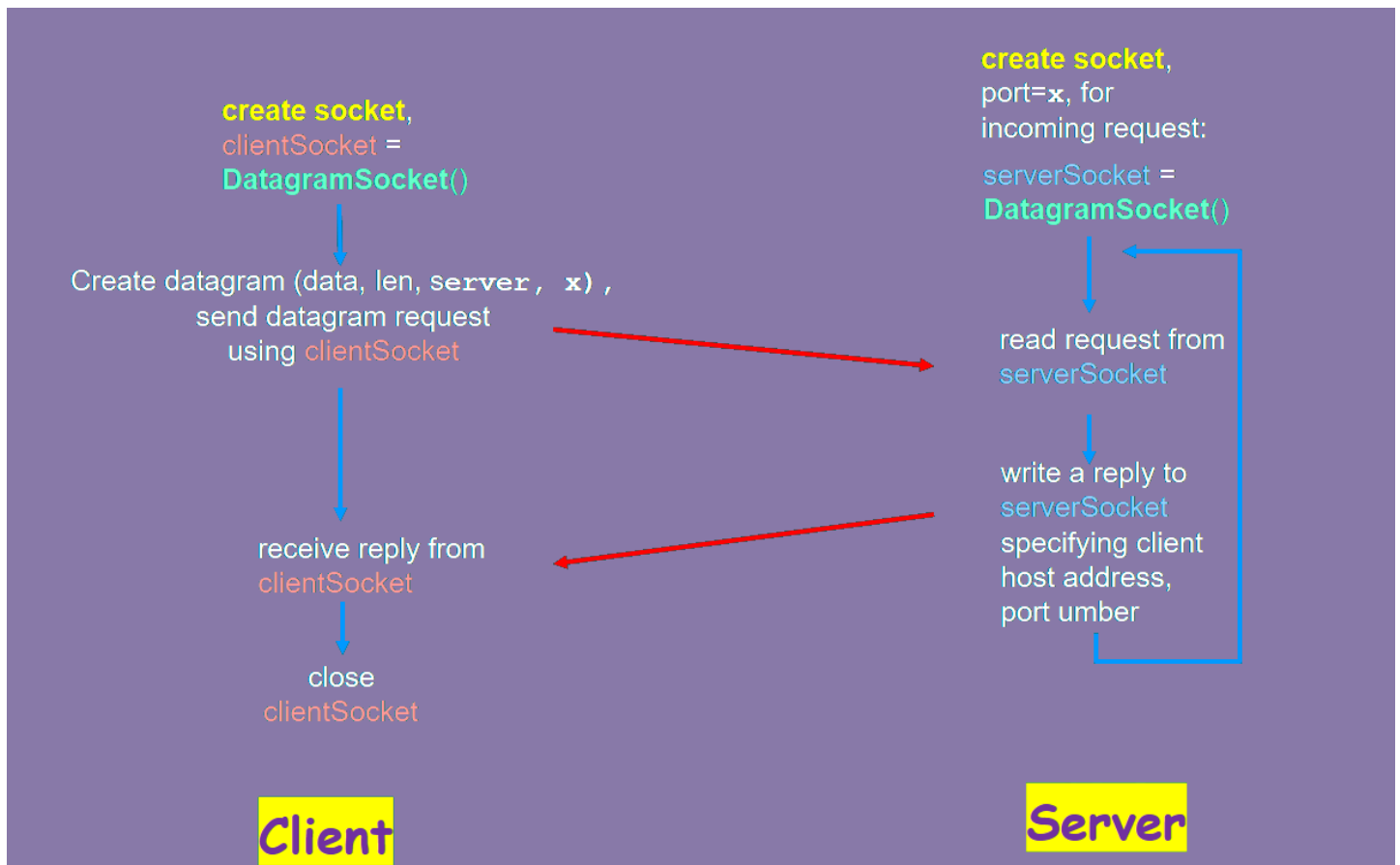
UDP payload (data)

1. لا يتعامل مع Stream of bytes.

2. نلاحظ ان حجم ال header هو 8 bytes.

3. حقل ال check sum يستخدم لاكتشاف الأخطاء أثناء النقل , قد تهمل الحزمة إذا وجد خطأ.

Client/Server Socket interaction



لا يوجد اتصال دائم بين ال Server وال client.

كل رسالة ترسل لوحدها داخل datagram.

ال server ينتظر الرسائل على رقم port وال client يرسل الرسالة مباشرة.

1- إنشاء Socket لل Server, يجب تحديد ال Port الذي سيكون فيه السيرفر بوضع listening لاستقبال الرسائل

```
DatagramSocket serverSocket = new DatagramSocket(x);
```

2- إنشاء Socket لل client, وهذا لا يحتاج لتحديد port او ip لأن النظام يختار له البورت تلقائياً، وهو ينشئ Datagram :

```
DatagramSocket clientSocket = new DatagramSocket();
```

3- إنشاء Datagram packets لإرسال البيانات من client :

```
Create datagram (data, data_length, serverIp, serverPort);
```

4- تصل الرسالة إلى serverSocket، يقرأها السيرفر، حيث وصل ال Datagram الى ال Port :

serverSocket.receive(packet);

5- السيرفر يرسل الرد لـ client عبر عنوانه الذي أتى منه ال Datagram، ثم ينشئ packet جديدة ويرسل الرد.

6- يكون العميل منتظراً حتى تصله رسالة السيرفر:

clientSocket.receive(packet);

7- يغلق clientSocket ويبقى ال ServerSocket ينتظر رسائل من clients آخرين.

The UDP Classes

Java.net.DatagramPacket	Java.net.DatagramSocket
هو Wrapper (غلاف) لمصفوفة بايتات الداتا التي سيتم ارسالها او استلامها	(Socket) هو بورت الاتصال UDP الخاص بروتوكول
مصفوفة البيانات طول البيانات عنوان الجهاز الذي سترسل اليه رقم بورت الجهاز المستقبل	يرسل Datagram Packets يستقبل Datagram Packets يتصل ب Port معين يتوصل مع عدة Ip Addresses
Public Datagram Packet (byte[] data, int lengths, Inet Address host, int port)	Public Datagram Socket (int port, Inet Address addr) Throws Socket Exception. port : رقم البورت الذي سيستقبل السوكيت به. addr : Ip الجهاز الذي نريد ربطه بالبورت. Socket Exception : يرميه عند حدوث مشكلة في الشبكة أثناء محاولة الموصول للسوكيت.

بعض الملاحظات عن Exceptions :

بعض الاستثناءات المرتبطة بال Sockets :

java.io.IOException:

استثناء عام يشير لوجود مشكلة بالدخل والخرج ومنها مشاكل الشبكات.

java.net.SocketException:

فيها IOException وهنا يدل على مشكلة في التعامل مع socket سواء UDP أو TCP .

: اسباب حدوث Socket Exception

- .1 No Route to Host Exception
- .2 Timeout
- .3 Socket Closed
- .4 Connect Exception
- .5 Bind Exception
- .6 Connection Refused
- .7 Connection Reset
- .8 Broken Pipe

Example 1: DayTime Protocol Client:

```
private static final String host = "time.nist.gov";

try (
    //1
    DatagramSocket socket = new DatagramSocket(){
    //2
    socket.setSoTimeout(10000);
    //3
    InetAddress host = InetAddress.getByName("time.nist.gov");
    //4
    DatagramPacket request = new DatagramPacket(new byte[1], 1, host, 13);
    //5
    socket.send(request);
    //6
    DatagramPacket response = new DatagramPacket(new
        byte[1024], 1024);
    //7
    socket.receive(response);
    //8
    String result = new String(response.getData(), 0,
        response.getLength(), "US-ASCII");
    System.out.println(result); }
```

يرسل ال Client طلباً إلى ال Server time.nist.gov.

1- تم إنشاء Socket

```
try (DatagramSocket socket = new DatagramSocket());
```

تم إنشاء Datagram Socket دون رقم بورت حيث يختار النظام أي منفذ حر (غير متصل بأي جهاز) تلقائياً، ويستخدم للإرسال والاستقبال عبر UDP.

2- socket. SetSoTimecut (10000)

تحديد مهلة الانتظار 10 ثواني لاستقبال الرد ، إذا لم يصل شيء خلال هذه الفترة يرمي Socket let Time out Exception.

الحصول على IP السيرفر (DNS look up), إذا فشل تحويل ال hostname إلى ip يعطي Un known Host Exception .

4- إنشاء Datagram Packet الإرسال (نشئة إما للاستقبال أو للإرسال) ، في TCP لا يوجد مخاطب على مستوى ال Application حيث يجب إنشاء اتصال أولاً ، أما هنا يوجد اتصال على مستوى ال Application ، حيث يرسل إلى client للسيرفر أي شيء طوله 1 Byte فقط.

5- إرسال الطلب

6- إنشاء Data gram للاستقبال ، حيث يتم تجهيز Buffter بسعة 1024 لاستقبال الرد.

7- استقبال الرد

8- تحويل شكل البيانات إلى string.



Example 1: DayTime Protocol Server

```

import java.net.*;
import java.util.Date;
import java.io.*;

try (
    //1
    DatagramSocket socket = new DatagramSocket(13)) {
    //2
    while (true) {
        try {
            //3
            DatagramPacket request = new DatagramPacket(new
                byte[1024], 1024);
            //4
            socket.receive(request);
            //5
            String daytime = new Date().toString();
            byte[] data = daytime.getBytes("US-ASCII");
            //6
            DatagramPacket response = new DatagramPacket(data, data.length, request.getAddress(),
                request.getPort());
            //7
            socket.send(response); } } }

```

يمثل خادم Daytime بسيط جداً ، يقوم بتشغيل خدمة الوقت على البورت 13.

1- هنا أنشأنا Socket السيرفر ، وعرفنا أنها للسيرفر لأننا مررنا لها رقم البورت ، فأني client سيرسل UDP إلى السيرفر ستصل البورت 13.

2- حلقة لا نهائية ليبقى السيرفر قيد التشغيل ليستقبل الطلبات .

3- تحضير Datagram packet لتستقبل الرسالة ، حيث هيئنا Buffer بحجم 1024 bytes.

4- استلام رسالة ال client التي تم وضعها في ال Datagram request التي أنشأناها به (وهو الحصول على الوقت).

5- الحصول على الوقت الحالي من النظام وتحويله المصفوفة bytes لأن Datagram packet يتعامل فقط مع مصفوفة بايتات ، وذلك تجهيزاً لإرسال الرد.

6- إنشاء Datagram packet للرد ، ترسل الداتا التي طولها data.length إلى عنوان ال العميل الذي يتم جلبه من request.getAddress() و البورت الخاص به من request.getPort().

7- إرسال الرد الذي تم وضعه به response.

The Datagram Socket Class

يستخدم لإرسال واستقبال البيانات باستخدام UDP .

Constructors:

public DatagramSocket(int port, InetAddress laddr) throws SocketException	public DatagramSocket() Throws Socket Exception يفتح Socket على port محلي مجهول (غير محدد)
public DatagramSocket (SocketAddress interface) throws Socket Exception	public DatagramSocket (int port) Throws Socket Exception
يفتح Socket على Port محدد ليستمع الى اتصالات شبكة محلية محددة.	يفتح Socket على Port محدد ليستمع الى كل الاتصالات الشبكة المحلية .

مثلاً:

```
Socket Address address = new InetAddress("127.0.0.1", 9999);
```

```
DatagramSocket socket = new DatagramSocket (address);
```

Methods:

```
public void send (Datagram Packet dp) throws IOException
```

```
Socket.send (request);
```

```
public void receive (Datagram packet dp) throws IOException
```

```
Socket.receive (response);
```

```
closing the Socket: public void close();
```

```
Socket.close();
```

وذلك حفاظاً على الموارد وتحريرها

```
public int getLocalPort()
```

يعيد البورت المحلي يعمل عليه السوكيت

```
public InetAddress getLocalAddress()
```

يعيد InetAddress object فيه العنوان الخاص بالجهاز الذي يعمل عليه الـ Datagram Socket اي العنوان المرتبط بالسوكيت .

```
public void connect (InetAddress host, int port)
```

UDP لا ينشئ اتصال حقيقي مثل TCP ، لكن جافا تسمح له بإنشاء pseudo connection باستخدام هذا التابع هذا يعني أن السوكيت سيرسل ويستقبل على هذا العنوان فقط.
بعد أن يتم الاتصال ، يمكننا استخدام الدوال التالية :

```
public int getport()
```

إذا كان السوكيت متصل يعيد رقم البورت البعيد (من الجهة الأخرى) ، أما إذا لم يكن متصل فيعيد -1.

```
public InetAddress getInetAddress()
```

إذا كان السوكيت متصل يعيد IP Address الطرف الآخر وإذا لم يكن متصلاً يعيد null.

```
public SocketAddress getRemoteSocketAddress()
```

إذا كان السوكيت متصل يعيد Socket Address object فيه (Ip,port) وإذا لم يكن متصل يعيد null .

The Datagram Packet class

في بروتوكول UDP كل رسالة ترسل عبر الشبكة تمثل packet (حزمة بيانات). وفي جافا يتم تمثيل هذه الحزم باستخدام الكلاس java.net.DatagramPacket. في هذا الكلاس يوجد 6 constructors بشكل عام، منها:
2 مخصصة للاستقبال (receiving).
4 مخصصة للإرسال (sending).
جميع الـ constructors تشترك في أنها تعتمد على مصفوفة بايتات من النوع: byte[] buffer
لان UDP يرسل البيانات كبايتات وليس String او Object ، وطول البيانات ايضا يتم تمريره بكل استدعاء لاي Constructor.

The constructors for receiving

```
public DatagramPacket(byte[] buffer, int length)
```

يتم تخزين الداتا في buffer ابتداء من buffer[0] حتى نهاية الـ length.

```
public DatagramPacket(byte[] buffer, int offset, int length)
```

يبدأ تخزين البيانات في المصفوفة ابتداء من buffer[offset]

The constructors for sending

هنا نحتاج إلى عنوان الوجهة:

```
public DatagramPacket(byte[] data, int length, InetAddress destination, int port)

public DatagramPacket(byte[] data, int offset, int length, InetAddress destination, int port)

public DatagramPacket(byte[] data, int length, SocketAddress destination)

SocketAddress تمثل ال InetAddress, و port معا.

public Datagram Packet (byte[] data, int offset, intLength, SocketAddress destination)
```

مثال:

```
String s = "This is a test";
byte[] data = s.getBytes("UTF-8");
try{
InetAddress ia = InetAddress.getByName("www.ibiblio.org")
int port = 7;
DatagramPacket dp = new DatagramPacket(data, data.length, ia, port);
} catch (IOException ex) {
handle exception //
}
```

هذا الكود ينشئ DatagramPacket فيها البيانات "this is a test" بترميز UTF-8 ، ويتم إرسالها إلى البورت 7 (المحجوز لسيرفر echo) أحد السيرفرات لدى www.ibiblio.org .

ملاحظة :

عند الاستقبال لا تعرف عنوان أو بورت المرسل مسبقاً ولكن تضاف تلقائياً عند وصول الحزمة .
عند الإرسال تحتاج لتحديد عنوان الوجهة.

The get Methods

`public InetAddress getAddress()`

تعطي عنوان الـ IP للمرسل أو للمستقبل.

`public int getPort()`

تعطي رقم البورت للمرسل أو المستقبل.

`public SocketAddress getSocketAddress()`

تعطي العنوان عامل للمرسل أو المستقبل (Ip+port).

`public int getLength()`

تعطي طول البيانات الفعلية الموجودة في الحزمة.

`public int getOffset()`

تعطي البداية الفعلية للبيانات داخل مصفوفة البايتات.

Example 3:

```
//1
String s = "This is a test.";
try {
    byte[] data = s.getBytes("UTF-8");

//2
    InetAddress ia =
        InetAddress.getByAddress("www.ibiblio.org");

//3
    DatagramPacket dp= new DatagramPacket(data, data.length, ia,
        7);

//4
    System.out.println("This packet is addressed to "+
        dp.getAddress() + " on port " + dp.getPort());
    System.out.println("There are " + dp.getLength() + " bytes of
        data in the packet");
    System.out.println(new String(dp.getData(), dp.getOffset(),
        dp.getLength(), "UTF-8"));
} catch (UnknownHostException |
    UnsupportedEncodingException ex){

}}}
```

- 1- إنشاء نص بسيط وهو البيانات التي سيتم ارسالها في UDP، ثم تحويله إلى بيانات.
- 2- الحصول على عنوان ال ip الخاص ب www.ibiblio.org.
- 3- إنشاء packet الإرسال مع عنوان المستقبل و رقم البورت.
- 4- طباعة المعلومات.

Java Threads

توجد طريقتين لتمثيل ال Thread في لغة جافا :

Implementing Runnable Interface	Extending Thread Class
نجعل الكلاس يقوم ب implement من هذه ال interface , ونكتب كود الدالة run() داخله مما يعني تنفيذ محتواها عبر ال Threads , هذه الطريقة تسمح بأن يرث الكلاس من كلاس اخر لأن الجافا لا تدعم الوراثة المتعددة	يمكن جعل الكلاس الذي نريد أن نأخذ منه Thread أن يرث الكلاس Thread و يعمل Override للتابع run() الذي سيحوي الكود الذي سيتم تنفيذه خلال هذه ال Thread عند إنشاء Object منها , وهذه الطريقة لاتسمح بالوراثة من كلاس اخر

Extending Thread class

ننشئ كلاس يعطينا Thread بأن نجعله يرث من كلاس Thread بهذه الخطوات التالية :

1.Class name: className extends Thread

2.Override Thread's run() method

3.Create a thread:

```
className t = new className();
```

4.Call start() method to start execution of the thread.

Thread constructors:

Thread()

Thread(String name)

Thread(Runnable r)

Thread(Runnable r, String name)

Example :

```

public class MyThread1 extends Thread {
    @Override
    public void run() {
        System.out.println("Inside thread");
    } // end of run
    public static void main(String[] args) {
        System.out.println("Creating thread...");
        MyThread t = new MyThread1();
        System.out.println("Starting thread...");
        t.start();
    } // end of main
} // end of MyThread1

```

في الكود أعلاه نرى أنه جعلنا الكلاس MyThread يرث من Thread و Override للتابع run() حيث يطبع جملة Inside Thread.
وهنا لدينا 2 Threads وهما : t و t أل Main Thread تمت طباعة Creating thread اولاً ثم Starting Thread و عندما استدعينا t.start() هنا بدأت ال Thread التي انشأناها بالعمل وعندما يطبع InsideThread.

ملاحظة :

دائماً هناك Thread خاص بالبرنامج الذي يكون قيد التنفيذ وهو ثريد ال main.

Example 2:

```

public class MyThread2 extends Thread {
    @Override
    public void run() {
        System.out.println("Inside : " +
            Thread.currentThread().getName());
    } // end of run
    public static void main(String[] args) {
        System.out.println("Inside : " +
            Thread.currentThread().getName());
        System.out.println("Creating thread...");
        MyThread2 t = new MyThread2();
        System.out.println("Starting thread...");
        t.start();
    }
}

```

هنا تم طباعة Inside: main أولاً لأنه قلنا أن Thread البرنامج الاساسي هو main ثم يطبع Creating thread
ثم Starting thread وعند استدعاء start() يطبع الجملة الخاصة بالتابع run : Thread 0 : inside.

Implementing Runnable interface

ننشر كلاس يقوم بعمل Implement Runnable interface و نكتب الكود الذي نريد تنفيذه في التابع run() عند عمل override له. (وهذه خطوة إجبارية)

ملاحظة :

Interface هو كلاس مجرد تماماً (abstract class) أي أننا نعرف داخله التوابع وأي كلاس تقوم بعمل implement له يجب أن تضع body لهذه التوابع، و override.

Example 3:

```
public class MyThread3 implements Runnable {
    @Override
    public void run() {
        System.out.println("Inside thread");
    } // end of run
    public static void main(String[] args) {
        System.out.println("Creating thread...");
        MyThread3 myObject = new MyThread3();
        Thread t = new Thread(myObject);
        System.out.println("Starting thread...");
        t.start();
    } // end of main
} // end of MyThread3
```

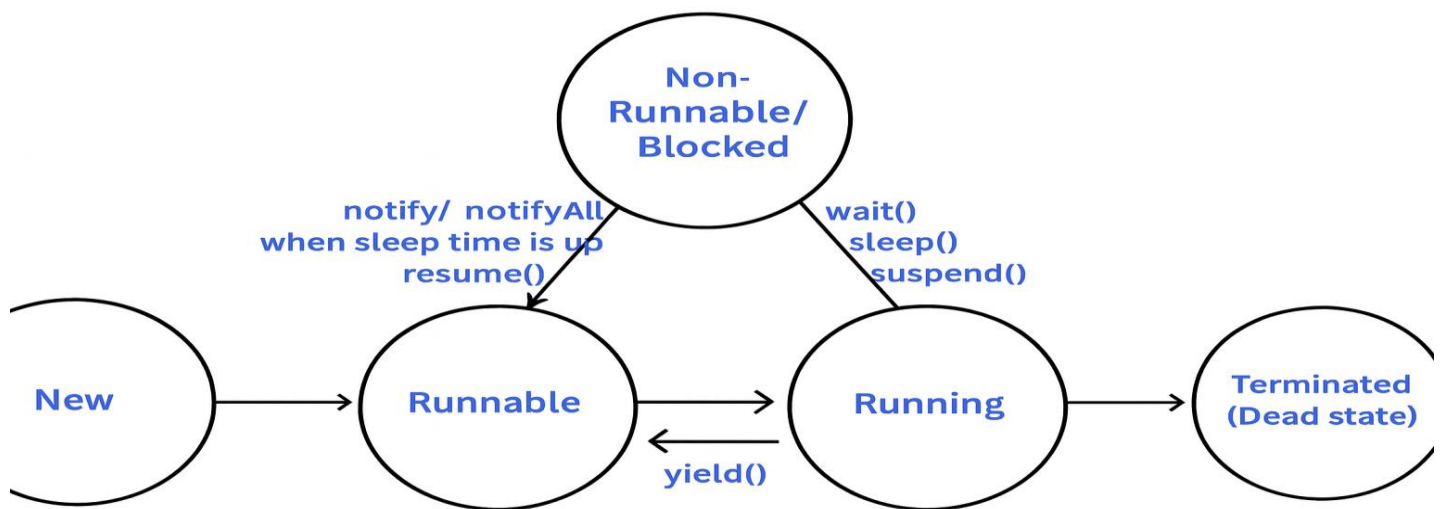
هنا أيضاً حسب ترتيب الـ Threads ، طبعاً لا ننسى أن البرنامج يبدأ التنفيذ عند تابع الـ main يطبع Creating thread :
ثم انشا اوبجكت من الكلاس MyThreads ووضعه داخل الـ Thread ثم طبع Starting Thread اعطى تعليمة البدء
للـ Thread فعندها تمت طباعة inside thread.

Which one is preferred:

يتم استخدام ال Runnable Interface عندما نريد الوصول لنفس الموارد من أكثر من object , وفي هذه الحالة استخدام الكلاس Thread يشكل عبء على الذاكرة و الأداء, لكن Implements Runnable لا تعطي Thread حيث يجب عمل object من Thread و تمرير ال instance إليه.

غالباً نستخدم ال Runnable عندما نريد عمل Override للتابع run() فقط وليس اي تابع اخر, عندما نريد توسيع Super class ونعلم انه لا يمكن عمل 2 extends لكلاس واحد, نستخدم ال interface ونعمل extends للكلاس الاخر.

Life Cycle Of Threads

أولاً :

- new : أي إنشاء لا object فقط دون البدء بالتنفيذ.
- runnable : مستعد للتنفيذ حيث تم إنشاؤه واستدعاء start().

ثانياً :

- running : يتم تنفيذ ال Thread بعد أن أُعطيَ أولوية وتم استدعاء run() , لكن يمكن أن يحدث أثناء التنفيذ :
 1. I/O request : تعليمات تحتاج إلى ال I/O المتاح
 2. yield() : تتوقف عملية تشغيل ال thread الحالي, لكي تُنفذ threads أخرى.
 3. sleep() : يتوقف التنفيذ لزمان محدد يُمرّر للتابع هنا ويُقاس بالميلي ثانية.
 4. join() : تجعل thread ما ينتظر لحين انتهاء thread ثانية أو أمر ما, ويمكن تحرير الوقت بعدها.

في هذه الحالات الأربعة تدخل حالة Block لحين الانتهاء من مدة الإيقاف ثم تعود إلى runnable إما تنتهي المدة المعطاة أو تُعطي notify أو تكتمل عليه I/O فيعود للتنفيذ.

ثالثاً :

عندما تعود لحالة runnable ثم running إما أن ينهي بعدها أو أن يعاود إيجاد عائق ما للانتهاء.

بعض التوابع التي يمكن استخدامها مع الـ thread :

-public int getPriority()

معرفة أولوية تنفيذ الـ thread.

-public void setPriority(int priority)

تغيير الأولوية لـ thread ما.

-public String getName()

إرجاع اسم الـ Thread.

-public void setName(String name)

تغيير اسم الـ thread.

ملاحظة :

priority من 1←10 و الـ default هو 5



Multi Thread Program

Example:

```
class A extends Thread {
    public void run(){
        for(int i=1;i<=5;i++){
            System.out.println("\t From ThreadA: i= "+i);
        }
        System.out.println("Exit from A");
    } // end of run & class A

class B extends Thread {
    public void run() {
        for(int j=1;j<=5;j++){
            System.out.println("\t From ThreadB: j= "+j);
        }
        System.out.println("Exit from B");
    } // end of run & class B

class C extends Thread {
    public void run(){
        for(int k=1;k<=5;k++){
            System.out.println("\t From ThreadC: k= "+k);
        }
        System.out.println("Exit from C");
    } // end of run & class C

class ThreadTest{
    public static void main(String args[]){
        new A().start();
        new B().start();
        new C().start();
    }
}
```

وسيكون الخرج كالتالي :

جعلنا الكلاسات class A و B و C، ترث من Thread، وتطبع بالترتيب:

From Thread A,B,C / i = 1 -->5

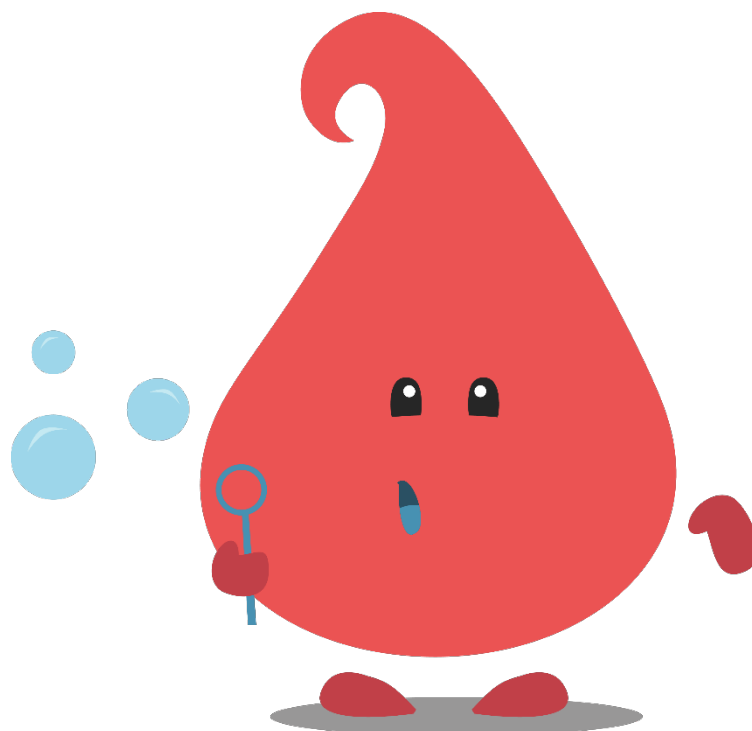
فكان الخرج للمرة الأولى:

//output1

```
From ThreadA: i= 1
From ThreadA: i= 2
From ThreadA: i= 3
From ThreadA: i= 4
From ThreadA: i= 5
Exit from A
From ThreadC: k= 1
From ThreadC: k= 2
From ThreadC: k= 3
From ThreadC: k= 4
From ThreadC: k= 5
Exit from C
From ThreadB: j= 1
From ThreadB: j= 2
From ThreadB: j= 3
From ThreadB: j= 4
From ThreadB: j= 5
Exit from B
```

لأننا بدأنا تشغيل كل Thread على حدى: A ثم B ثم C،

وكل واحد منهم مستقل عن الآخر، ففي المرة الأولى كل Thread أخذ وقته حتى انتهى تنفيذه كاملاً.



الخرج في المرة الثانية:

//output2

```

From ThreadA: i= 1
From ThreadA: i= 2
From ThreadA: i= 3
From ThreadA: i= 4
From ThreadA: i= 5
From ThreadC: k= 1
From ThreadB: j= 1
From ThreadC: k= 2
From ThreadC: k= 3
From ThreadC: k= 4
From ThreadC: k= 5
Exit from C
From ThreadB: j= 2
From ThreadB: j= 3
From ThreadB: j= 4
From ThreadB: j= 5
Exit from B
Exit from A

```

نلاحظ أن الخرج يختلف وتداخلت الـ Threads مع بعضها، وذلك بسبب أن Thread Scheduling غير حتمي (non-deterministic).

ولكن يوجد حل بحيث ننهي تنفيذ Thread كامل دون تداخل مع آخر، وهو Synchronization.

Synchronization:

تنظيم وصول الـ Threads للموارد المشتركة بحيث ينتظر كل واحد دوره حتى ينتهي الـ Thread الآخر، مما يمنع التداخل.

في هذه الحالة، تم توفير طريقتين لذلك:

Synchronized method.1

Synchronized block.2

Block Synchronization

إذا أردنا تنفيذ مجموعة أسطر معاً فنغلفها دون مقاطعة: `synchronized(obj)` حيث تنفذ دون مقاطعة الـ `obj` يمثل القفل (وهو كائن يستخدم لتنظيم الوصول فمثلاً عند دخول `Thread` إلى قسم متزامن فإنه يتم القفل عليه حتى ينتهي فيتحرر القفل). مثال:

```
synchronized(System.out) {
    for(int k=1; k<=5; k++) {
        ;System.out.println("\t from Thread: k="+k)
    }
}
```

الشرح: عندما يدخل `Thread` إلى هذا البلوك يحصل على القفل المرتبط بـ `System.out`.

أي `Thread` آخر يحاول الدخول إلى أي بلوك أو كود متزامن على نفس الكائن (`System.out`) سيبقى في حالة انتظار حتى ينتهي `Thread` و يُمرر القفل.

وهذا يتضمن عدم التداخل.

من الأفضل استخدام `object` قابل للوصول (مثل `this` or `System.out`) كقفل إذ يؤدي ذلك إلى تعقيد أو خطر حدوث `deadlock`. فمن الأفضل تعريف قفل خاص:

```
private final Object lock = new Object();
synchronized(lock) { // الكود الذي لا يمكن لأكثر من Thread تنفيذه بأمان }
```

Method Synchronization

```
private Writer out;
public synchronized void wr(String message){
    Date d = new Date();
    out.write(d.toString());
    out.write('\t');
    out.write(message);
}
```

```
out.write("\r\n");
}
```

تقوم بعمل synchronized لأن التابع wr ليس static، فإن أي Thread يحتاج لقفل على تلك الـ instance في نفس الوقت، لذلك فقط واحد يمكنه تنفيذ (wr) في نفس الوقت على تلك الـ instance.

Thread Priority

كل Thread في الجافا له أولوية تشير لمدى أهميته بالنسبة للمجدول scheduler الخاص بالـ Threads في الجهاز. الأولوية لا تضمن التنفيذ المتقدم تماماً، بل هي مثل توصية للـ (OS + JVM) حول أي Thread يأخذ وقتاً على المعالج أولاً.

نظام الـ Priority المعيارية:

Thread.MAX_PRIORITY = 10

Thread.NORM_PRIORITY = 5 (Default)

Thread.MIN_PRIORITY = 1

لا يوجد للأولوية قيمة أقل من 1، فإذا أعطينا (0) لـ setPriority سيعطي IllegalArgumentException.



Example:

```
class A extends Thread {
    public void run() {
        System.out.println("Thread A started");
        for(int i=1;i<=4;i++) {
            System.out.println("\t From ThreadA: i= "+i);
        }
        System.out.println("Exit from A"); }}

class B extends Thread {
    public void run(){
        System.out.println("Thread B started");
        for(int j=1;j<=4;j++) {
            System.out.println("\t From ThreadB: j= "+j);
        }
        System.out.println("Exit from B");
    }}

class C extends Thread {
    public void run(){
        System.out.println("Thread C started");
        for(int k=1;k<=4;k++) {
            System.out.println("\t From ThreadC: k= "+k);
        }
        System.out.println("Exit from C"); }}

class ThreadPriority {
    public static void main(String args[]) {
        A threadA=new A();
        B threadB=new B();
        C threadC=new C();
        threadC.setPriority(Thread.MAX_PRIORITY);
        threadB.setPriority(threadA.getPriority()+1);
        threadA.setPriority(Thread.MIN_PRIORITY);

        System.out.println("Started Thread A");
        threadA.start();

        System.out.println("Started Thread B");
        threadB.start();

        System.out.println("Started Thread C");
        threadC.start();

        System.out.println("End of main thread");
    }}
```

عرفنا Classes يرثون من Thread وفي كل واحدة يتم طباعة من 14 ثم يطبع

Thread A: 1

Thread B: 6

Thread C: 10

ثم أنشأنا 3 Thread من كل class وأعطينا أولوية Thread كالتالي:

ثم عند البدء بالتشغيل والتنفيذ، يكون الخرج عملياً:

مضمون ال Thread C

مضمون ال Thread B

مضمون ال Thread A

ولكن يعتمد ذلك أيضاً على الـ Os Type، Os version، JVM version .

THE END